

PM 2023

Rattrapage (Remy MACLE)

1 Sujet 104

Ce rattrapage est un travail individuel dans le même esprit que le projet original de première session. Il s'agit de concevoir (en C) le programme de simulation numérique d'un système décrit par une équation différentielle non linéaire du deuxième ordre. La méthode d'intégration numérique imposée est RK4 classique (la même qu'en première session).

1.1 Équation

L'équation à intégrer est la suivante :

$$\frac{d^2y}{dt^2} + \delta \frac{dy}{dt} + \alpha y + \beta y^3 = \gamma \cos(\omega t) \quad (1)$$

2 Spécifications

2.1 Valeurs numériques et conditions initiales

Les unités ne sont pas précisées, il suffit d'admettre l'homogénéité de l'équation avec les valeurs numériques suivantes :

- $\alpha = -1,00$
- $\beta = 1,00$
- $\delta = 0,20$
- $\gamma = 1,20$
- $\omega = 1,80$

Les conditions initiales (y_0 et y'_0) seront définies au moment du lancement du programme de résolution (sous la forme d'arguments de la ligne de commande).

2.2 Fourniture

Par courriel, à votre adresse institutionnelle (remy.macle@uha.fr), vous sera envoyée une archive (format zip). L'extraction de cette archive créera un dossier de même nom (`simu104`) contenant trois fichiers :

- `Makefile` : ce fichier vous permet de vérifier comment sera compilé votre fichier source ;
- `simu104.c` : ce fichier est le squelette initial de votre fichier source, à compléter ;
- `simu104.pdf` le présent sujet.

2.3 Livrable

Votre programme de résolution sera un fichier source C unique nommé « `simu104.c` ».

ATTENTION : pour des raisons de portabilité, vous êtes limité à l'utilisation des en-têtes définis par la norme SUS4. Pour faire simple, limitez-vous aux en-têtes (bien suffisants) déjà présents dans le squelette fourni (vous êtes libre d'en rajouter mais n'en supprimez aucun).

Sur un système Unix (certifié SUS4), votre source doit impérativement pouvoir être compilé sans erreur (ni *warning*) par la commande suivante :

```
$ make
```

Cette commande exploite le fichier `Makefile` présent dans l'archive fournie. Il n'y a pas à modifier ce `Makefile`.

À retenir et respecter absolument :

- votre livrable se composera uniquement du seul fichier `simu104.c`,
- ne rajouter aucun fichier source supplémentaire ni `.h` ni `.c`,
- ne pas modifier le `Makefile`.

2.4 Exécution

Au moment de l'exécution, le programme devra admettre quatre arguments numériques, dans l'ordre :

- la durée totale de simulation (en secondes),
- le pas de calcul ou période d'échantillonnage (en secondes),
- la valeur initiale (condition initiale) de y (y_0)
- la valeur initiale (condition initiale) de y' (y'_0)

Par exemple (sur un système Unix ou compatible),

```
$ ./simu104 20 0,01 0,5 1,25
```

lancera une simulation du système pour une durée totale de 20 secondes, un pas de calcul d'un centième de seconde pour les conditions initiales $y_0=0,5$ et $y'_0=1,25$.

2.5 Présentation des résultats

Le programme devra afficher sur la sortie standard (`stdout`), les résultats bruts de simulation sous la forme d'une matrice où

- la première ligne servira de ligne de titre (un label par colonne),
- les lignes suivantes représenteront les résultats numériques (notation scientifique) de simulation à raison d'un pas de calcul (à un instant donné) par ligne, chaque colonne sera une valeur numérique dans l'ordre :
 - le temps (valeur de l'instant du pas de calcul en secondes),
 - la valeur de y ,
 - la valeur de y' ($\frac{dy}{dt}$),
- le séparateur de colonnes sera le caractère de tabulation (`\t`), aussi bien pour la ligne de titre que pour les lignes suivantes.

Voici un exemple de résultat conforme au format attendu

```
Temps (s) .   y (anonyme) dy (anonyme)
0,0000e+00    #,####e+##  #,####e+##
0,0010e+00    #,####e+##  #,####e+##
```

```
0,0020e+00  #,####e+##  #,####e+##  
0,0030e+00  #,####e+##  #,####e+##  
...
```

Ce format permet une représentation graphique facile par simple redirection et importation dans un tableur (openoffice, libreoffice...).

ATTENTION, il ne sera toléré aucun autre message sur la sortie standard. Si vous désirez afficher d'autres informations, utilisez obligatoirement la sortie d'erreur (`stderr`). Les fonctions (ou plutôt `macros`) `debug` et `trace` présentes dans le squelette initial (cf. 3.1) sont spécifiquement dédiées à cet usage.

2.6 Localisation

La localisation française est exigée, c'est-à-dire que tous les nombres s'écrivent avec une virgule et non pas un point, c'est la raison pour laquelle, l'instruction

```
setlocale(LC_NUMERIC, "fr_FR");
```

doit absolument être la première instruction de votre programme (conformément au squelette fourni). En conséquence, tous les nombres (aussi bien en entrée qu'en sortie) seront au format naturel où la virgule sera le séparateur entre partie entière et partie décimale.

En cas d'erreur, sur un système du type Linux, vérifiez que les paquets `locales` et `locales-all` sont bien installés.

Pour faire simple : ne touchez pas aux lignes 61 à 79 du squelette fourni, la localisation y est traitée correctement selon la plateforme de développement choisie.

3 Divers

3.1 Fournitures

Une archive (zip) vous sera fournie. Cette archive contiendra le squelette de développement à partir duquel vous pourrez développer votre propre projet. Ce squelette est explicitement destiné à un système du type Unix. Si vous ne disposez que d'un système Windows, le mieux sera d'utiliser une machine virtuelle, le développement natif sur Windows sera à vos risques et périls. Vous êtes prévenu que la vérification se fera obligatoirement sur un système du type Unix.

L'archive contient les quatre fichiers suivants :

- `simu104.c`, véritable squelette de départ de votre programme;
- le `Makefile` qui sera utilisé lors de la correction (commande `make`, paragraphe 2.3)

3.2 Sources externes

Vous êtes libres d'utiliser les ressources documentaires que vous voulez, soyez cependant prudent. N'oubliez jamais de citer vos sources.

4 Notifications importantes

4.1 Respect des spécifications

ATTENTION, le strict respect des spécifications fonctionnelles du type :

- nom du programme source et de l'exécutable,
- ordre et format des arguments,

- ordre et format des valeurs résultantes,
- localisation,

sont obligatoires. Votre travail sera évalué par un script automatique qui ne fera pas de sentiment (somme cumulée des erreurs quadratiques par rapport à un programme de référence pour un jeu arbitraire d'arguments).

4.2 Travail individuel

Attention, il s'agit bien d'un travail individuel, surtout n'échangez jamais le moindre extrait de code avec quiconque. Un contrôle anti-plagiat sévère fera partie de la validation de votre code.

4.3 Délai de livraison

Votre travail devra être transmis par courriel :

- à l'adresse : `denis.lienhardt@uha.fr`
- ne devra contenir que votre fichier `simu104.c` en pièce jointe
- dernier délai : aujourd'hui (jeudi 28 juin) à 14:00.

Comme lors du projet, vous êtes libre de me soumettre votre travail à plusieurs reprises pour en vérifier le fonctionnement. Seul le dernier soumis sera finalement conservé.

4.4 Rappel RK4

La méthode de Runge-Kutta d'ordre quatre avec dérivée seconde [\[modifier | modifier le code \]](#)

Dans le cas $y'' = f(t, y, y')$, $y(t_0) = y_0$, $y'(t_0) = y'_0$, nous pouvons décomposer le problème en un système d'équations :

$$\begin{cases} y' = g(t, y, z) \\ z' = s(t, y, z) \end{cases}$$

avec ici $y' = g(t, y, z) = z$ et $s = f$.

En appliquant la méthode RK4 à chacune de ces équations, puis en simplifiant nous obtenons¹ :

$$\begin{aligned} k_1 &= f(t_n, y_n, y'_n) \\ k_2 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}y'_n, y'_n + \frac{h}{2}k_1\right) \\ k_3 &= f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}y'_n + \frac{h^2}{4}k_1, y'_n + \frac{h}{2}k_2\right) \\ k_4 &= f\left(t_n + h, y_n + hy'_n + \frac{h^2}{2}k_2, y'_n + hk_3\right) \end{aligned}$$

On en déduit y_{n+1} et y'_{n+1} grâce à :

$$\begin{aligned} y_{n+1} &= y_n + hy'_n + \frac{h^2}{6}(k_1 + k_2 + k_3) \\ y'_{n+1} &= y'_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \end{aligned}$$

FIGURE 1 – Reproduction d'un extrait de la page wikipedia